

Quickstart

Chiffrement ASCON-128 sur FPGA PynqZ2
pour acquisition securisee de signaux ECG

Ali AHNANI & Khalid ELKOUSSAMI • Encadrants : J.-B. Rigaud & R. Viera
Ecole des Mines de Saint-Etienne — Institut Mines-Telecom 2025–2026



Objectif. Ce document constitue un guide complet et autonome pour prendre en main la chaîne entière du projet : création de l'environnement Python, configuration et programmation du FPGA, communication UART, lancement du chiffrement ASCON-128 et exploitation des signaux ECG côté PC. **Avec ce seul fichier vous pouvez lancer le projet de bout en bout.**

1 Description du projet

Le projet met en œuvre un **chiffrement authentifié ASCON-128** sur **FPGA PynqZ2** afin de sécuriser la transmission de **signaux ECG**. La carte reçoit les paramètres cryptographiques et la trame ECG via **UART**, réalise le chiffrement matériel, puis renvoie le **ciphertext** et le **tag**. Côté PC, l'application Python effectue le déchiffrement, la validation et l'analyse du signal.

Chaîne fonctionnelle : Capteur ECG → PC Python (IACQ) $\xrightarrow{\text{UART}}$ FPGA / ASCON-128 → PC Déchiffrement → Analyse NeuroKit2 (BPM, SDNN, PQRST...)

2 Prerequis matériels et logiciels

- Carte **PynqZ2** programmée avec le bitstream ASCON-128, reliée au PC via USB/UART.
- **Python 3.10+** installé sur le PC hôte.
- **Vivado 2025.x** pour régénérer ou programmer le bitstream.
- Archive du projet décompressée (contient tous les **.py** et **data/xNorm.csv**).

1 Environnement Python

1.1 Creer et activer un environnement virtuel

Un environnement virtuel isole les dependances du projet. Toutes les commandes s'executent depuis le **dossier racine du projet**.

Windows (PowerShell / cmd)

```
python -m venv .venv
.venv\Scripts\activate
```

Linux / macOS (bash / zsh)

```
python3 -m venv .venv
source .venv/bin/activate
```

Une fois active, l'invite de commande affiche **(.venv)** en prefixe.

1.2 Installer les dependances depuis requirements.txt

```
pip install -r requirements.txt
```

Paquet	Version min.	Role
numpy	1.26	Calculs numeriques (tableaux, statistiques RR)
pandas	2.0	Chargement et manipulation des donnees CSV
scipy	1.11	Filtres Butterworth HPF/LPF sur le signal ECG
neurokit2	0.2.10	Analyse ECG : detection PQRST, metriques HRV
matplotlib	3.7	Visualisation temps reel et graphiques finaux
pytest	7.0	Execution des tests unitaires

Note : **pyserial** n'est pas dans **requirements.txt** car il est requis uniquement avec le **FPGA reel**. Si vous utilisez le materiel physique, installez-le separement :

```
pip install pyserial
```

1.3 Verifier l'installation

```
python -c "import numpy, pandas, scipy, neurokit2, matplotlib; print('OK')"
```

La commande doit afficher **OK** sans erreur.

2 Programmation du FPGA

2.1 Ouverture du projet Vivado

Decompresser l'archive du projet Vivado puis ouvrir le fichier **.xpr** dans **Vivado 2025.x**. Si le bitstream n'est pas present, le regenerer via **Flow > Generate Bitstream**.

2.2 Chargement sur la carte PynqZ2

1. Brancher la carte au PC (USB-UART + alimentation).
2. Dans Vivado : **Open Hardware Manager** → **Open Target** → **Auto Connect**.

3. Program Device — selectionner le fichier .bit.

Reperes materiels : apres programmation, la LED **DONE** s'allume. Le baudrate UART est configurable via les boutons embarques (par default 115 200 baud). Le reset doit rester inactif pendant toute la session.

2.3 Identifier le port serie

Windows

```
# Gestionnaire de peripheriques -> Ports (COM et LPT) -> ex. COM4
```

Linux / macOS

```
ls /dev/ttyUSB* # Linux
ls /dev/tty.usb* # macOS
```

3 Lancement du projet

3.1 Mode emulateur (sans FPGA physique)

```
python main.py --emulator
```

3.2 Mode FPGA reel

```
python main.py --port COM4 # Windows
python main.py --port /dev/ttyUSB0 # Linux
python main.py --port /dev/tty.usbserial-XXXX # macOS
```

3.3 Options disponibles

Option	Description
-port COM4	Port serie du FPGA (default : COM4)
-baud 115200	Debit UART en bauds (default : 115 200)
-timeout 2	Delai de lecture en secondes (default : 2)
-emulator	Emulateur logiciel, sans FPGA physique
-trame-file data/xNorm.csv	Fichier CSV des trames ECG
-no-plot	Desactive la visualisation graphique

Resultat attendu : pour chaque trame, le terminal affiche **[OK]** avec tag et BPM. La fenetre graphique montre 3 courbes : original (vert), dechiffre + PQRST (bleu), chiffre (rouge).

4 Protocole UART — 7 commandes

Les commandes sont envoyees octet par octet. Le FPGA repond **OK\n** (4F 4B 0A) apres chaque commande valide.

Cmd	Hex	Taille	Role
K	0x4B	16 octets	Cle secrete ASCON-128
N	0x4E	16 octets	Nonce (unique par trame)
A	0x41	8 octets	Donnees associees (b"A to B" + 0x8000)
W	0x57	184 octets	Trame ECG (181 oct. + bourrage 0x800000)
G	0x47	—	Declenchement du chiffrement
T	0x54	16+3 oct.	Lecture du tag d'authentification
C	0x43	181+6 oct.	Lecture du ciphertext

5 Description des fichiers Python

5.1 main.py — Point d'entree principal

Orchestre la totalite du pipeline : parsing des arguments, chargement des trames, connexion FPGA, boucle de chiffrement, visualisation temps reel et rapport final.

Pseudocode — main.py

```

PARSE arguments (port, baud, emulator, trame-file, no-plot)
SETUP logger -> logs/fpga_comm.log

trames <- LOAD_TRAMES(trame-file) // liste de bytes[181]
IF trames empty -> EXIT(1)

iacq <- IACQ(port, baud, emulator=flag)
iacq.OPEN()
IF --no-plot: INIT matplotlib live figure (3 subplots)

FOR each trame IN trames:
    cipher, tag <- iacq.ENCRYPT_ON_FPGA(trame, KEY, NONCE, AD)
    plain <- iacq.DECRYPT_WAVEFORM(cipher, tag, KEY, NONCE, AD)
    ASSERT plain == trame
    IF plot: UPDATE live curves (original / decrypted+PQRST / ciphertext)
    LOG cycle time, BPM, statut

iacq.CLOSE()
PRINT rapport (avg_time, throughput, erreurs)
IF plot: SHOW analyse ECG finale

```

5.2 iacq.py — Interface UART / classe IACQ

Encapsule toute la communication serie avec le FPGA (ou l'emulateur). Gere la connexion, l'envoi des parametres, le cycle de chiffrement complet, la recuperation des resultats et le dechiffrement Python.

Pseudocode — iacq.py (classe IACQ)

```

CLASS IACQ:
  INIT(port, baud, timeout, emulator, max_retries)

  OPEN():
    IF emulator: connection <- FPGAEmulator()
    ELSE: connection <- serial.Serial(port, baud)

  SEND_COMMAND(cmd_byte, payload=[]):
    FOR attempt IN 1..max_retries:
      connection.write(cmd_byte + payload)
      response <- connection.read(200)
      IF response ends with "OK\n" -> RETURN response
      ELSE: WAIT 2s, RETRY
    RAISE FGATimeoutError

  ENCRYPT_ON_FPGA(waveform, key, nonce, ad):
    SEND_KEY(key) // 0x4B 16 bytes
    SEND_NONCE(nonce) // 0x4E 16 bytes
    SEND_ASSOCIATED_DATA(ad) // 0x41 8 bytes
    SEND_WAVEFORM(wf+0x800000)// 0x57 184 bytes
    START_ENCRYPTION() // 0x47
    tag <- GET_TAG() // 0x54, resp[:3]
    ciphertext <- GET_CIPHertext() // 0x43, resp[:6]
    RETURN (ciphertext, tag)

  DECRYPT_WAVEFORM(cipher, tag, key, nonce, ad):
    plain <- ascon_decrypt(key, nonce, ad, cipher + tag)
    IF plain == None -> RAISE FGAAAuthenticationError
    RETURN plain

```

5.3 fpga_emulator.py — Emulateur logiciel du FPGA

Simule exactement le comportement UART du FPGA reel. Utilise `ascon_pcsn.py` pour chiffrer en interne. Sert de bouchon de test sans materiel.

Pseudocode — fpga_emulator.py (FPGAEmulator)

```

CLASS FPGAEmulator:
  WRITE(data):
    cmd <- data[0]; payload <- data[1:]
    response <- PROCESS_COMMAND(cmd, payload)
    PUSH response -> read_buffer

  PROCESS_COMMAND(cmd, payload):
    0x4B (K): key <- payload[0:16] -> "OK\n"
    0x4E (N): nonce <- payload[0:16] -> "OK\n"
    0x41 (A): ad <- strip_padding(payload) -> "OK\n"
    0x57 (W): plaintext <- payload[0:181] -> "OK\n"
    0x47 (G):
      result <- ascon_encrypt(key, nonce, ad, plain)
      ciphertext <- result[:-16]
      tag <- result[-16:] -> "OK\n"
    0x54 (T): RETURN tag + "OK\n"
    0x43 (C): RETURN ciphertext + 0x800000 + "OK\n"

  READ(n) / READLINE():
    POP n bytes from read_buffer (avec timeout)

```

5.4 ascon_pcsn.py — Implementation de reference ASCON-128

Algorithme ASCON-128 AEAD en Python pur (standard NIST 2023). Utilise pour le dechiffrement PC et par l'emulateur.

Pseudocode — ascon_pcsn.py

```
FUNCTION ascon_encrypt(key, nonce, ad, plaintext):
  state <- INITIALIZE(key, nonce) // Permutation P12
  state <- PROCESS_AD(state, ad) // Absorb AD + P6
  cipher, state <- ENCRYPT_BLOCKS(state, plaintext) // P6/bloc
  tag <- FINALIZE(state, key) // P12 -> tag[128b]
  RETURN cipher + tag

FUNCTION ascon_decrypt(key, nonce, ad, cipher_with_tag):
  cipher <- cipher_with_tag[:-16]
  tag_rcv <- cipher_with_tag[-16:]
  state <- INITIALIZE(key, nonce)
  state <- PROCESS_AD(state, ad)
  plain, state <- DECRYPT_BLOCKS(state, cipher)
  tag_calc <- FINALIZE(state, key)
  IF tag_calc != tag_rcv -> RETURN None // echec auth
  RETURN plain
```

5.5 visualization.py — Visualisation des signaux ECG

Trace les signaux ECG avec analyse cardiaque complete : delineation PQRST via NeuroKit2, metriques HRV et detection d'anomalies.

Pseudocode — visualization.py

```
FUNCTION plot_ecg_data(data_df, title, save_path):
  // data_df colonnes : original | plain | cipher
  CREATE figure 4 subplots:
    [1] Signal original (vert)
    [2] Signal dechiffre (bleu) :
      ecg_clean <- nk.ecg_clean(plain, fs=360)
      rpeaks <- nk.ecg_peaks(ecg_clean)
      waves <- nk.ecg_delineate(ecg_clean, rpeaks)
      SCATTER pics P/Q/R/S/T
      metrics <- COMPUTE_HRV(rpeaks)
    [3] Ciphertext (rouge)
    [4] Rapport texte : BPM, SDNN, RMSSD, pNN50 + anomalies

FUNCTION compute_hrv_metrics(rpeaks, fs=360):
  rr <- diff(rpeaks)/fs * 1000 // intervalles RR ms
  RETURN bpm=60000/mean(rr), sdnn=std(rr),
    rmssd=sqrt(mean(diff(rr)^2)), pnn50=%rr>50ms

FUNCTION detect_anomalies(metrics):
  IF bpm < 50 -> "Bradycardie"
  IF bpm > 100 -> "Tachycardie"
  IF sdnn < 20 -> "HRV faible (dysfonction autonome)"
  IF rmssd > 50 OR pnn50 > 50 -> "Rythme irregulier"
```

5.6 demo_ascon.py — Demo autonome du chiffrement

Script standalone sans FPGA : charge une trame ECG depuis CSV, chiffre et dechiffre avec ASCON, genere [demo_plot.png](#).

Pseudocode — demo_ascon.py

```
key <- fromhex("8A55114D1CB6A9A2BE263D4D7AECAAFF")
nonce <- fromhex("4ED0EC0B98C529B7C8CDDF37BCD0284A")
ad <- b"A to B"

waveform <- LOAD_FROM_CSV("data/xNorm.csv", line=0)
// Lit la ligne 0 (hex) -> bytes[181]

payload <- ascon_encrypt(key, nonce, ad, waveform)
ciphertext <- payload[:-16]
tag <- payload[-16:]
decrypted <- ascon_decrypt(key, nonce, ad, payload)
ASSERT decrypted == waveform // validation bit-a-bit

PLOT 3 courbes : original (bleu) / ciphertext (rouge) / decrypted (vert)
SAVE "demo_plot.png"
```

5.7 utils.py — Configuration du journal

Configure le logging double (fichier + console) partagé par tous les modules.

Pseudocode — utils.py

```
FUNCTION setup_logger(log_file="logs/fpga_comm.log"):
    MKDIR(dirname(log_file))
    logger <- getLogger("IACQ_LOGGER")
    IF aucun handler existant:
        ADD FileHandler(log_file, level=DEBUG)
        ADD StreamHandler(console, level=INFO)
    format <- "%(asctime)s - %(levelname)s - %(message)s"
    RETURN logger
```

5.8 exceptions.py — Hierarchie des exceptions

Distingue les erreurs de protocole, connexion, authentification et validation.

```
FPGAError
|- FPGAConnectionError # connexion impossible ou perdue
|- FGATimeoutError # pas de reponse dans le delai
|- FPGAValidationError # donnees invalides (taille, format)
|- FPGAProtocolError # reponse malformee (sans "OK\n")
|- FPGAAuthenticationError # tag ASCON incorrect

IACQError
|- EncryptionError # echec chiffrement / dechiffrement
|- DataValidationError # CSV invalide
```

6 Conseils de validation et debogage

- **Pas d'acquiescement OK\n** : verifier le baudrate (carte et script), et que le reset est inactif.
- **Tag incorrect** : verifier que **KEY** et **NONCE** dans **main.py** correspondent exactement au firmware du FPGA.
- **Trame rejetee** : la trame doit faire exactement **181 octets** avant bourrage.
- **Timeout UART** : augmenter **-timeout** (default 2s) ou reduire le debit.

- **Test sans FPGA** : `python main.py -emulator` — resultat garanti identique.
- **Logs details** : consulter `fpga_communication.log` et `fpga_emulator.log`.

Validation bit-a-bit : si `decrypted == original` pour chaque trame, la chaine ASCON-128 est fonctionnelle de bout en bout. La visualisation finale indique **Rythme sinusal normal** ou **Anomalie detectee** selon les metriques HRV calculees.

Projet FPGA Securite — Ecole des Mines de Saint-Etienne

Ali AHNANI • Khalid ELKOUSSAMI | Encadrants : Jean-Baptiste Rigaud & Raphael Viera